From Cole: To the Rest

The Complete LipScribe Technical Handbook: Everything Achieved, Every Setback Met, and Every Decision Taken (Weeks 1–4)

Cole Rodney Migisha	b29657	Charis Opol	b29654
Ainomugisha Faith	b27518	Lisa Lusinga Namugenyi	b30300
Ansasire Nehum Naamaye	b29785	Wakisa Isabelle	b30311

Project: LipScribe – Lip-Assisted Speech Recognition for Robust Live Captioning 9 November 2026

A Note from Cole to Charis, Faith, Lisa, Nehum, and Isabelle

This booklet is written as a complete, self-contained tutor guide so that every member of Group 4 has the exact same deep understanding of our entire system—from raw video and audio files in Week 1 all the way to our state-of-the-art **Proposed Model P** in Week 4. Nothing is skipped: every mathematical intuition, every bug and setback we hit, why it happened, how we fixed it, every graph, and every terminal command you need to run the codebase yourself is right here.

32,890
Verified GRID clips across 33 speakers (96.7 % pass rate)

1.02 %
Best greedy clean test WER (Model P4; 0.71 % with grammar)

1.88 %
Model P4 no-video fallback WER (vs. 25.18 % for B3 v2)

30.9 %
Visual lip gate weight at −5 dB (up from 19.6 % clean)

Contents

1 Chapter 1 — The Big Picture: What Problem Are We Solving (and How CTC Works)? 2
- 1.1 Why Audio-Only Recognition Fails in Noise and Why Lips Alone Are Not Enough 2
- 1.2 The 75-Frame Temporal Synchronisation Contract (3.0 s = 75 Frames) 2
- 1.3 How CTC (Connectionist Temporal Classification) Works 3

2 Chapter 2 — Week 1: Data Integrity, End-Pointed Audio, Speaker Splits, and Mouth Cropping 3
- 2.1 Setbacks #1 & #2: What `verify_dataset.py` Uncovered in the Raw GRID Corpus 3
- 2.2 Speaker-Independent Splits (`splits.csv`) & Disjoint MUSAN Noise Pools (`noise_splits.csv`) 4
- 2.3 Setback #3: MediaPipe Mouth-Cropping Failure Handling (`lipscribe/preprocessing/crop.py`) 4

3 Chapter 3 — Week 2: The 5 Integration Bugs Caught Before Training 5

4 Chapter 4 — Week 3 (Part 1): Baselines B1 & B2 and the "`bin blue at f sive snon`" Overfit Mystery 5
- 4.1 Baseline B1: Audio-Only (`configs/b1_audio_only.yaml`) 5
- 4.2 Setback #9: The B2 Visual-Only Overfit Mystery ("`bin blue at f sive snon`") . . 5
- 4.3 Full-Corpus B2 Visual-Only (V1 `conv3d_bilstm`) & Setback #10 6

5 Chapter 5 — Week 3 (Part 2): Baseline B3, V2 Upgrades (`ResNet-18`), and Grammar Decoding **6**
5.1 Setback #11: Why B3 v1 (Trained From Scratch) Helped in Noise But Hurt Clean Audio 6
5.2 The Four V2 Upgrades That Solved Setbacks #10 and #11 7

6 Chapter 6 — Week 4: Proposed Model P (Cross-Modal Attention, Gated Fusion, `e_miss`, and Full Results) **7**
6.1 How Proposed Model P (`LipScribeModel`) Works 7
6.2 Full-Corpus 4-Way Model P Experiment (`P1`, `P2`, `P3`, `P4`) 8

7 Chapter 7 — Master Scoreboard & Complete Log of All 13 Setbacks **9**

8 Chapter 8 — Team Command Cheat-Sheet: Run Everything Yourself **10**

---

1 Chapter 1 — The Big Picture: What Problem Are We Solving (and How CTC Works)?

1.1 Why Audio-Only Recognition Fails in Noise and Why Lips Alone Are Not Enough

When someone speaks in a quiet room (**Clean** condition), an audio-only speech recogniser easily separates vowels and consonants from the spectrogram. However, as background noise increases—measured as **Signal-to-Noise Ratio (SNR)** dropping from Clean $\rightarrow$ 10 dB $\rightarrow$ 5 dB $\rightarrow$ 0 dB $\rightarrow$ -5 dB (where -5 dB means background noise power is **3.16× stronger** than the voice!)—acoustic formants and high-frequency consonant bursts (`/f/`, `/s/`, `/b/`, `/p/`) are buried.

Visual lip movements are 100 % immune to acoustic noise: whether the room is silent or at -5 dB, the mouth pixels are identical. Conversely, lipreading alone struggles with **homophenes**—distinct sounds produced with virtually identical lip shapes (e.g., bilabials `/b/`, `/p/`, `/m/` or alveolars `/d/`, `/t/`, `/n/`). Therefore:

- **Audio** provides voicing and nasality (separating `/b/` from `/p/` and `/m/`).
- **Video (Lips)** provides place of articulation and lip aperture (separating `/b/` from `/f/` or `/s/` even when noise drowns the microphone).

1.2 The 75-Frame Temporal Synchronisation Contract (3.0 s = 75 Frames)

Every clip in the GRID corpus lasts 3.0 seconds at 25 frames per second (fps), giving $T = 75$ video frames (one frame every 40 ms):

- **Video stream:** 25 fps $\times$ 3.0 s $=$ 75 grayscale mouth frames of size 96×96 $\Longrightarrow$ $[B, 1, 75, 96, 96]$.
- **Audio stream:** 16,000 Hz waveform (48,000 samples) converted to 80-band log-Mel spectrograms using a 25 ms Hann window and 10 ms hop (100 frames/s) $\Longrightarrow$ $[B, 300, 80]$.
- **Exact $4:1$ alignment:** Notice that $300 \div 4 = 75$. In `lipscribe/models/audio_encoder.py`, two `Conv1d` layers each with stride $= 2$ downsample time by $2 \times 2 = 4$ ($[B, 300, 80] \rightarrow [B, 150, 256] \rightarrow [B, 75, 256]$), followed by a 2-layer BiLSTM outputting $[B, 75, 512]$.
- Meanwhile, our visual encoder (`VisualEncoder` or `ResNetVisualEncoder`) uses temporal stride 1, outputting $[B, 75, 512]$. Thus at every frame index $t \in \{1, \ldots, 75\}$, the audio vector

$\mathbf{a}_t \in \mathbb{R}^{512}$ and visual vector $\mathbf{v}_t \in \mathbb{R}^{512}$ describe the **exact same 40 ms slice of speech**.

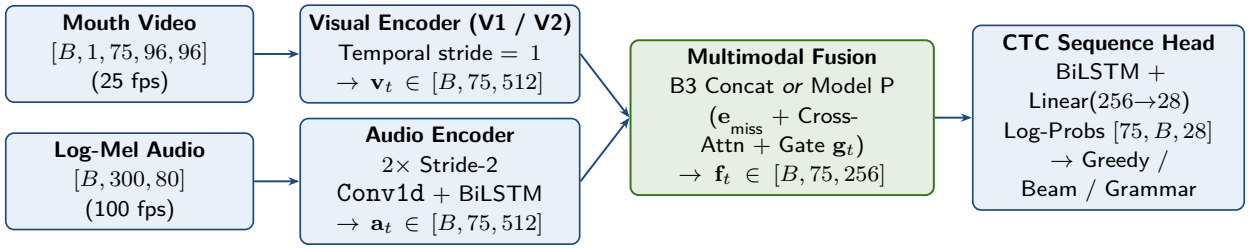


Figure 1: End-to-end LipScribe tensor flow and the 75-frame (25 fps) alignment contract fixed in `docs/INTERFACES.md`.

1.3 How CTC (Connectionist Temporal Classification) Works

We have $T = 75$ frames, whereas a target transcript like `"bin blue at f two now"` has only $L = 21$ characters, and we do not know which exact frame corresponds to each letter. **CTC** solves alignment-free sequence learning by adding a special `<blank>` symbol (index `0`) to our character vocabulary (`space = 1`, `a..z = 2..27`), giving **28 classes**:

1. **Step 1 (Merge consecutive duplicates):** `[b, b, 0, i, i, n, 0, n]` → `[b, 0, i, n, 0, n]`.
2. **Step 2 (Remove all blanks `0`):** `[b, 0, i, n, 0, n]` → `"binn"`.
3. **Why blank is required for double letters:** In words like `"green"` or `"soon"`, emitting `[e, e]` collapses to a single `"e"`. To output `"ee"`, the network *must* emit `[e, 0, e]`. Consequently, a transcript of length L with R repeated adjacent characters requires at least $L + R$ frames. Across all 33,000 GRID transcripts, the longest sentence is 31 characters with at most 3 adjacent repeats ($31 + 3 = 34 \leq 75$), so $T = 75$ never violates the CTC length condition.

2 Chapter 2 — Week 1: Data Integrity, End-Pointed Audio, Speaker Splits, and Mouth Cropping

2.1 Setbacks #1 & #2: What `verify_dataset.py` Uncovered in the Raw GRID Corpus

Before training anything, Faith wrote `scripts/verify_dataset.py` (Card **W1-07**) to audit all 34,000 raw GRID utterances across 7 checks. This immediately caught two major data traps:

- **Setback #1 (Missing Speaker `s21` & Corrupted Clips):** Speaker `s21` had audio files only (zero videos or alignments); speaker `s8` had 108 undecodable videos; `s4` had 1 bad video (`bbbm2n`); `s28` had 1 bad audio clip. **Decision:** Excluded `s21` and filtered out the 110 corrupted clips, leaving **32,890 verified utterances across 33 speakers (96.7 % pass rate)**.
- **Setback #2 (End-Pointed Audio Desynchronisation):** While every video lasted 3.0 s (75 frames), the raw GRID `.wav` files were *end-pointed* (trimmed to only the spoken portion, lasting 1.10–2.94 s, mean 1.83 s), whereas speakers sat silently for an average of 0.47 s (up to 1.51 s) into the video before speaking! If we placed the audio at $t = 0$, the voice would play 0.5–1.5 s **ahead of the lips**. **Decision:** In `lipscribe/data/dataset.py`, we read the exact word-onset timestamp from each utterance's `.align` file and shift the audio waveform into a 3.0 s (48,000-sample) buffer at the exact frame the lips begin moving.

2.2 Speaker-Independent Splits (`splits.csv`) & Disjoint MUSAN Noise Pools (`noise_splits.csv`)

To guarantee zero speaker or noise leakage:

- **Speaker split (`data/splits.csv`, seed 2026): Train**: 26 speakers (14M, 12F; 25,891 clips). **Validation**: 3 unseen speakers (`s11`, `s19`, `s26`; 3,000 clips). **Test**: 4 unseen speakers (`s4`, `s9`, `s14`, `s24`; 2M, 2F; 3,998 usable clips).
- **MUSAN noise split (`data/noise_splits.csv`):** Split *by noise file* (70 % train / 15 % val / 15 % test) so test noise files are never heard during training.

2.3 Setback #3: MediaPipe Mouth-Cropping Failure Handling (`lipscribe/preprocessing/crop.py`)

In Cards **W1-10** and **W2-01**, I built the mouth-cropping pipeline using MediaPipe Face Mesh (`FACEMESH_LIPS` landmarks, square bounding box +25 % margin, smoothed with a centred 5-frame moving average, resized to 96×96 grayscale `uint8`).

- **Setback #3 (What if Face Mesh misses a frame?):** If MediaPipe fails on 1–2 frames and we drop those frames, T shrinks from 75 to 73 and breaks audio–video alignment.
- **Decision: Never drop frames.** Failed frames linearly interpolate their crop bounding box from the nearest valid frames before and after, while recording `valid[t] = False` in the per-frame validity mask (Figure 2). Clips with $> 10\,\%$ invalid frames are excluded in `build_manifest.py`.

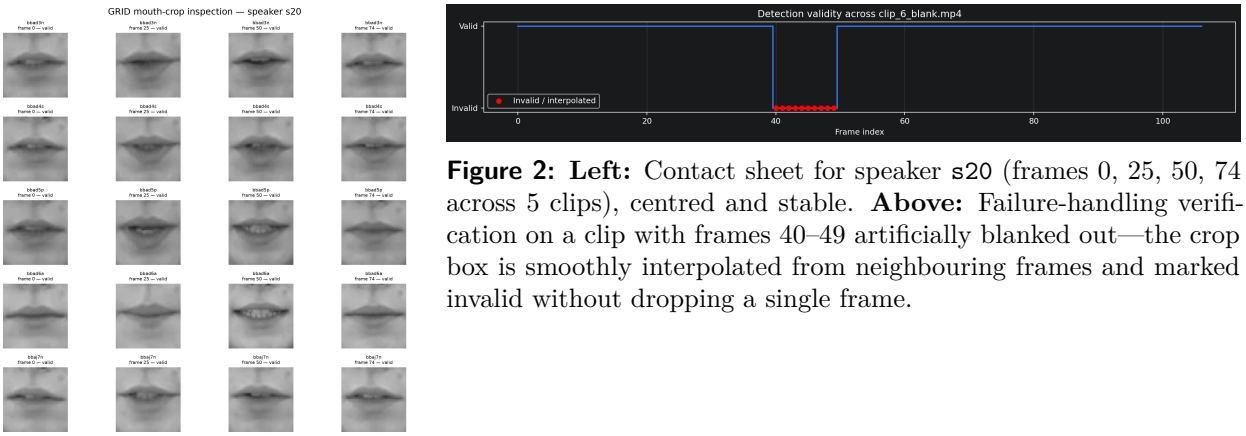


Figure 2: Left: Contact sheet for speaker `s20` (frames 0, 25, 50, 74 across 5 clips), centred and stable. **Above:** Failure-handling verification on a clip with frames 40–49 artificially blanked out—the crop box is smoothly interpolated from neighbouring frames and marked invalid without dropping a single frame.

Listing 1: Week 1–2 Data Verification, Mouth Extraction, and Audio Preparation Commands

```
# 1. Audit all 34,000 GRID utterances and write reports/integrity_report.csv:
.venv/bin/python -m scripts.verify_dataset

# 2. Extract 96x96 mouth crops in parallel across CPU cores (resumable):
.venv/bin/python -m scripts.extract_mouths --workers 8

# 3. Standardise audio to 16 kHz mono WAV & generate fixed noisy val/test sets:
.venv/bin/python -m scripts.prepare_audio
.venv/bin/python -m scripts.make_noisy_test

# 4. Build master manifest.csv and training-only feature_norm.npz stats:
.venv/bin/python -m scripts.build_manifest
.venv/bin/python -m scripts.compute_norm_stats
```

3 Chapter 3 — Week 2: The 5 Integration Bugs Caught Before Training

Every individual module passed its unit tests in isolation, but when Charis and I wired the end-to-end training loop together in Week 2, we uncovered and fixed **five critical integration bugs**:

1. **Setback #4 (Alignment Parser Reading Timestamps Instead of Words):** GRID `.align` lines have the format `start_time end_time word`. An early parser indexed token `0` instead of token `2`, turning transcripts into numbers. Fixed in `lipscribe/data/tokenizer.py` and verified on all 33,000 files.
2. **Setback #5 (End-Pointed Audio Offset):** Fixed by shifting trimmed audio waveforms according to the `.align` speech-start timestamp (PR #21).
3. **Setback #6 (DataLoader Workers Cloning SNR Sequences):** When PyTorch forked `num_workers` processes, every worker inherited the exact same NumPy/Python RNG seed and sampled identical noise SNRs. Fixed by reseeding each worker per epoch via `seed + epoch * 1000 + worker_id` (PR #21).
4. **Setback #7 (Loudness Varying with SNR):** Level-normalising to −25 dBFS *before* adding noise made −5 dB clips much louder than clean clips. Moved −25 dBFS RMS normalisation (gain capped at +20 dB) to run *after* noise mixing (PR #21).
5. **Setback #8 (`.gitignore` Hiding `lipscribe/models/`):** An unanchored `models/` rule in `.gitignore` (meant for top-level MediaPipe weights) hid `lipscribe/models/` from Git. Anchored to `/models/`.

4 Chapter 4 — Week 3 (Part 1): Baselines B1 & B2 and the "`bin blue at f sive snon`" Overfit Mystery

4.1 Baseline B1: Audio-Only (`configs/b1_audio_only.yaml`)

Charis trained **B1** (`AudioEncoder` + `CTCHead`, 3.55 M parameters) for **52 epochs** (best Epoch 43, ≈ 76.5 s/epoch on our RTX 4080 SUPER).

- **Official Held-Out Test WER: 1.96 % (clean)**, 3.02 % (10 dB), 3.79 % (5 dB), 5.44 % (0 dB), and **9.45 % (−5 dB)**.
- **Takeaway:** B1 is excellent in quiet conditions (1.96 %), but its word error rate increases nearly **5-fold** at −5 dB.

4.2 Setback #9: The B2 Visual-Only Overfit Mystery ("`bin blue at f sive snon`")

Before launching a multi-hour full-corpus visual-only run (**B2**, Card **W3-05**), our protocol required overfitting 8 utterances to CTC Loss < 0.05 and CER < 0.10. Across our first 6 runs in `scripts/check_b2_overfit.py`, the model refused to converge! Here is what we discovered:

1. **Runs 1–2 (Synthetic Gaussian White Noise Collapse):** Passing `torch.randn` tensors through three stages of 3D convolution, BatchNorm, and MaxPool acted as a spatial low-pass filter. Feature vectors from 8 independent random clips had **87 % cosine similarity**, trapping CTC loss at ≈ 1.21.
2. **Runs 3–6 (The "`bin blue at f sive snon`" Prefix Mode Collapse):** When we loaded the first 8 real clips sequentially (`ds[0]..ds[7]`), the loss froze at **0.5940** and CER stuck at exactly **0.2804**. Printing the predictions revealed that the network output the exact same string for all 8 clips:

```
[0] Ref: 'bin blue at f two now'      -> Hyp: 'bin blue at f sive snon'
[1] Ref: 'bin blue at f three soon'   -> Hyp: 'bin blue at f sive snon'
[2] Ref: 'bin blue at f four please'  -> Hyp: 'bin blue at f sive snon'
[3] Ref: 'bin blue at f five again'   -> Hyp: 'bin blue at f sive snon'
[4] Ref: 'bin blue at l six now'      -> Hyp: 'bin blue at f sive snon'
```

Why? Because `manifest.csv` is sorted alphabetically (`bbaf2n`, `bbaf3s`...), the first 8 clips all shared the 14-character prefix `"bin blue at f "`! The BiLSTM memorised the shared prefix without using the 3D-CNN at all, and for the differing suffixes (`two/three/four/five/six/seven` + `now/soon/please/again`), it predicted the character-level centroid average `"sive snon"`—which matches 71.96 % of characters ($1 - 0.7196 = 0.2804$ CER), trapping the gradients in a deep local minimum!

3. **Runs 7–9 (The Fix — Unigram-Diverse Sampling & Phase Transition):** We modified `load_overfit_batch()` to sample 8 clips with distinct starting verbs (`bin`, `lay`, `place`, `set`) and colours (`blue`, `green`, `red`, `white`). With zero shared prefix, the 3D-CNN was forced to learn real lip shapes from frame 1. At step 2,860, a phase transition occurred and loss plunged from 0.76 → **0.0497** with **CER** = 0.0000 **(100 % exact character match across all 8 clips)**!

4.3 Full-Corpus B2 Visual-Only (V1 `conv3d_bilstm`) & Setback #10

Full-corpus **B2 v1** trained for 28 epochs (best Epoch 19) and achieved **23.04 % greedy test WER** (14.03 % CER; **21.03 %** with grammar decoding) across all five SNRs.

- **Setback #10 (2.5× Unseen Speaker Gap):** Per-speaker test WER varied from 13.3 % (`s24`) to 33.7 % (`s9`). Why? Because V1's three plain `Conv3d` blocks flatten a $96 \times 6 \times 6 = 3{,}456$ map into a dense `Linear(3456, 512)` layer (1.77 M params), tying weights to rigid (x, y) pixel coordinates and overfitting to training speaker face geometry and lighting.

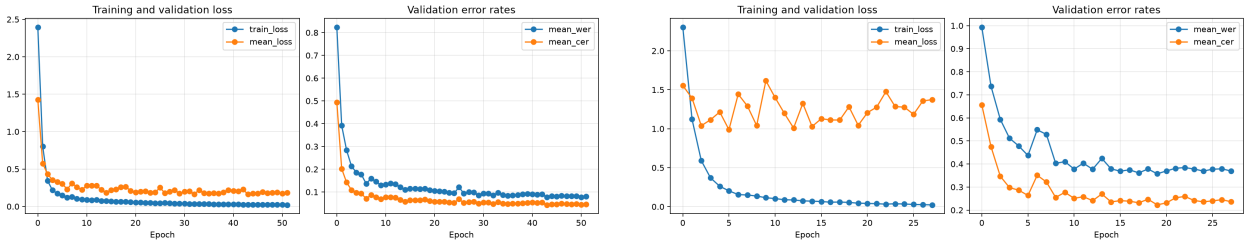


Figure 3: Left: B1 Audio-Only training and validation curves (52 epochs). **Right:** B2 Visual-Only V1 training and validation curves (28 epochs).

5 Chapter 5 — Week 3 (Part 2): Baseline B3, V2 Upgrades (`ResNet-18`), and Grammar Decoding

5.1 Setback #11: Why B3 v1 (Trained From Scratch) Helped in Noise But Hurt Clean Audio

Faith implemented `B3Concat` (`lipscribe/models/fusion_concat.py`), which concatenates $[\mathbf{a}_t; \mathbf{v}_t] \in \mathbb{R}^{1024}$ and projects to 256-d. When trained from scratch for 22 epochs (**B3 v1**):

- At −5 dB, B3 v1 beat B1 (7.35 % vs. 9.45 % WER — a 22.2 % relative reduction).
- **Setback #11:** On **clean audio**, B3 v1 (2.41 %) was slightly *worse* than audio-only B1 (1.96 %). Because the audio encoder converges in ∼ 10 epochs while the visual encoder needs 25+ epochs from scratch, the undertrained visual branch injected noise into clean audio

predictions while the dominant audio gradients starved the visual encoder.

5.2 The Four V2 Upgrades That Solved Setbacks #10 and #11

1. **V2 Hybrid 3D-Stem + 2D ResNet-18 Visual Encoder (`resnet18_bilstm`):**
   - **Per-clip photometric normalisation (`input_norm="clip"`):** Standardises each clip across its valid frames to zero mean and unit variance before the first convolution, eliminating per-speaker lighting and contrast bias.
   - **5-frame 3D spatiotemporal stem (`Conv3d` 5×7×7):** Captures 200 ms co-articulatory lip motion at the input without 3D FLOP explosion in deeper layers.
   - **16-layer 2D ResNet-18 residual trunk + `AdaptiveAvgPool2d(1)`:** Replaces the coordinate-tied 3,456→512 flatten bottleneck with deep residual blocks and global spatial pooling, enforcing translation invariance.
   - **Stronger augmentation:** Scale jitter (±10 %), spatial cutout, time masking (two masks up to 6 frames), and `Dropout2d(0.2)`.
2. **Pre-trained Encoder Initialisation (`init_from`):** Initialise multimodal models directly from the converged `B1` and `B2` checkpoints.
3. **Modality Dropout:** Randomly zero out the audio stream (p_a) or visual stream (p_v) per training utterance so the fusion layer must learn from both modalities.
4. **Exact GRID Grammar Viterbi Decoder (`lipscribe/decoding/grammar.py`):** Alongside Isabelle's CTC Prefix Beam Search (17.29 ms/utt), we built a vectorised Viterbi decoder over the exact 64,000-sentence GRID grammar trie (11.0 ms/utt on CPU, zero retraining required).

Result in B3 v2: Initialising from `B1 + B2 v2` with modality dropout cut B3 v2's greedy test WER to **1.16 % (clean)** and **5.21 % (−5 dB)**—**0.77 % (clean)** and **3.83 % (−5 dB)** with grammar decoding!

- **Setback #12 (B3 v2's Fatal Unimodal Collapse):** When we tested B3 v2 with one modality removed at test time (`--drop-modality visual` or `--drop-modality audio`), B3 v2 collapsed to **25.18 % WER without video** and **75.31 % WER without audio** because linear concatenation has no dynamic gate or learned missing-modality representation.

6 Chapter 6 — Week 4: Proposed Model P (Cross-Modal Attention, Gated Fusion, `e_miss`, and Full Results)

6.1 How Proposed Model P (`LipScribeModel`) Works

Implemented in `lipscribe/models/fusion_attention.py` (Cards **W4-01** to **W4-04**), **Model P** introduces three mechanisms that solve B3's limitations:

1. **Learned Missing-Modality Embedding ($\mathbf{e}_{\text{miss}} \in \mathbb{R}^{256}$):** *Setback #12b:* If you mask all keys in `nn.MultiheadAttention` when a stream is missing, softmax$(-\infty, \ldots, -\infty)$ produces `NaN`! Instead, whenever a stream is dropped (during training modality dropout $p_a = 0.25, p_v = 0.15$, test-time `--drop-modality`, or live inference when MediaPipe detects a face in $< 50\,\%$ of frames), we replace that stream's projected features with the learned vector $\mathbf{e}_{\text{miss}}$.
2. **Bidirectional 4-Head Cross-Modal Attention (`CrossModalAttention`):** Audio queries Video ($\tilde{\mathbf{a}} = \text{LN}(\mathbf{a}_{\text{proj}} + \text{MHA}(Q = \mathbf{a}_{\text{proj}}, K = \mathbf{v}_{\text{proj}}, V = \mathbf{v}_{\text{proj}}))$) and Video queries Audio ($\tilde{\mathbf{v}} = \text{LN}(\mathbf{v}_{\text{proj}} + \text{MHA}(Q = \mathbf{v}_{\text{proj}}, K = \mathbf{a}_{\text{proj}}, V = \mathbf{a}_{\text{proj}}))$).

3. **Per-Frame Sigmoid Gated Fusion (`GatedFusion`):**

$$\mathbf{g}_t = \sigma(\mathbf{W}_g[\tilde{\mathbf{a}}_t; \tilde{\mathbf{v}}_t] + \mathbf{b}_g) \in (0,1)^{256}, \qquad \mathbf{f}_t = \mathbf{g}_t \odot \tilde{\mathbf{a}}_t + (1 - \mathbf{g}_t) \odot \tilde{\mathbf{v}}_t$$

6.2 Full-Corpus 4-Way Model P Experiment (`P1`, `P2`, `P3`, `P4`)

We trained all four initialisation variants of Model P on the **full GRID corpus** (30-minute wall-clock budget per variant on the RTX 4080 SUPER, matching B3 v2):

- **P1 (`p_scratch`):** Audio and visual (`Conv3D`) trained from scratch (11 epochs).
- **P2 (`p_init_b2`):** Visual initialised from `B2 v1` (`Conv3D`); audio random (11 epochs).
- **P3 (`p_init_b1_b2`):** Audio from `B1` + visual from `B2 v1` (`Conv3D`) (10 epochs).
- **P4 (`p_init_b1_b2_v2` / `p_proposed`):** Audio from `B1` + visual from `B2 v2` (`ResNet-18`) (6 epochs).

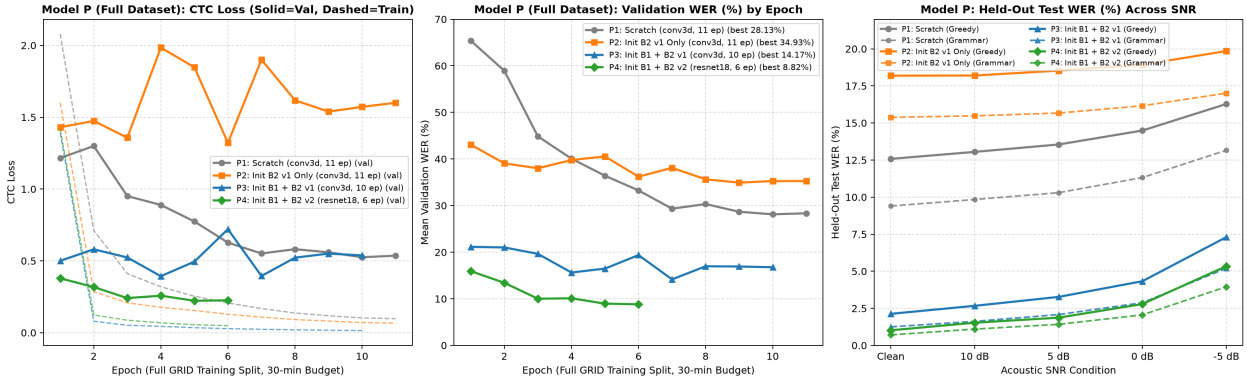


Figure 4: Full-corpus 4-way Model P comparison (`P1--P4`): CTC loss convergence (left), validation WER trajectory by epoch (centre), and official held-out test WER across all 5 SNRs for greedy and grammar decoding (right).

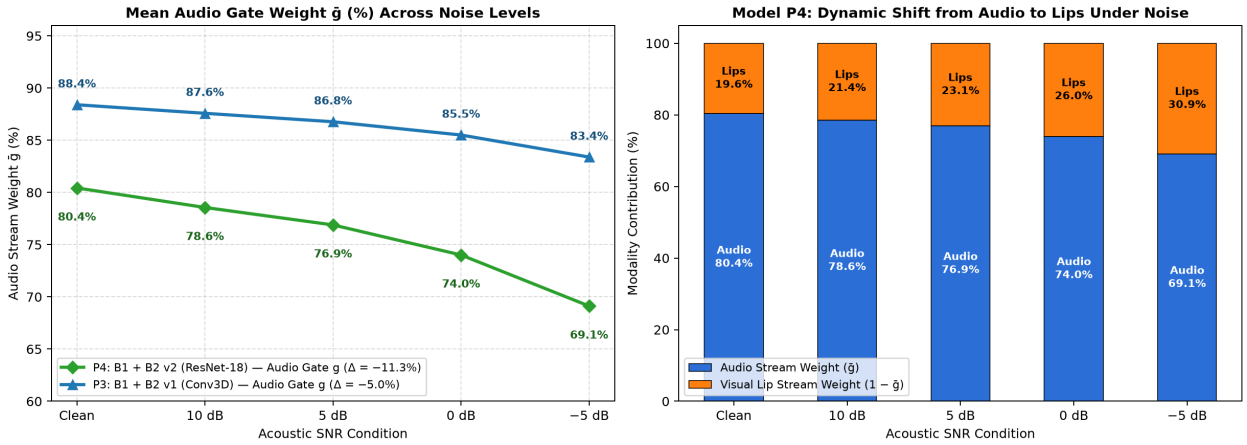


Figure 5: Proof of Dynamic Noise-Adaptive Gating: As acoustic noise increases from Clean to −5 dB, Model P4 monotonically lowers its audio gate $\bar{g}$ from 80.4 % down to 69.1 % ($\Delta\bar{g} = -11.3$ %, 2.25× stronger than P3's `Conv3D` gate shift), increasing visual lip reliance from 19.6 % up to 30.9 %.

7 Chapter 7 — Master Scoreboard & Complete Log of All 13 Setbacks

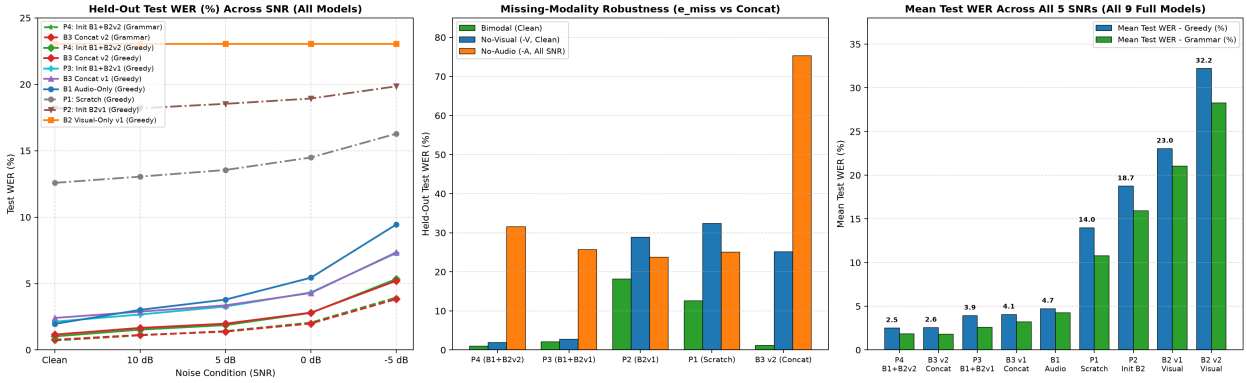


Figure 6: Master comparison across all 9 full-corpus models: Held-out test WER vs. SNR (left), missing-modality robustness comparing $\mathbf{e}_{\text{miss}}$ against concatenation (centre), and mean test WER across all 5 SNRs (right).

Table 1: Master Scoreboard: Official Held-Out Test WER % Across All 9 Full-Corpus Trained Models (3,998 test utterances per SNR).

Model	Visual / Init	Val WER	Clean Grd/Grm	10 dB Grd/Grm	5 dB Grd/Grm	0 dB Grd/Grm	−5 dB Grd/Grm	Mean Grd/Grm	No-Aud (−A)	No-Vis (−V Cln)
B1 Audio-Only	— / Scratch	7.72	1.96 / 1.76	3.02 / 2.79	3.79 / 3.47	5.44 / 4.93	9.45 / 8.35	4.73 / 4.26	—	1.96
B2 Visual v1	Conv3D / Scr	35.74	23.04 / 21.03	23.04 / 21.03	23.04 / 21.03	23.04 / 21.03	23.04 / 21.03	23.04 / 21.03	23.04	—
B2 Visual v2	Res18 / Scr	42.38	32.20 / 28.26	32.20 / 28.26	32.20 / 28.26	32.20 / 28.26	32.20 / 28.26	32.20 / 28.26	32.20	—
B3 Concat v1	Conv3D / Scr	12.37	2.41 / 1.86	2.88 / 2.27	3.36 / 2.65	4.29 / 3.45	7.35 / 5.87	4.06 / 3.22	—	—
B3 Concat v2	Res18 / B1+B2v2	8.72	1.16 / 0.77	1.66 / 1.13	1.98 / **1.38**	2.81 / **1.99**	**5.21 / 3.83**	2.56 / **1.82**	75.31	25.18
Model P1	Conv3D / Scr	28.13	12.58 / 9.40	13.05 / 9.84	13.55 / 10.30	14.49 / 11.32	16.28 / 13.16	13.99 / 10.80	25.03	32.41
Model P2	Conv3D / B2v1	34.93	18.19 / 15.38	18.20 / 15.48	18.53 / 15.67	18.93 / 16.16	19.85 / 17.01	18.74 / 15.94	**23.78**	28.82
Model P3	Conv3D / B1+B2v1	14.17	2.13 / 1.25	2.67 / 1.62	3.27 / 2.08	4.33 / 2.88	7.30 / 5.20	3.94 / 2.61	25.65	2.72
Model P4 (Best)	Res18 / B1+B2v2	**8.82**	**1.02 / 0.71**	**1.53 / 1.10**	**1.87** / 1.42	**2.79** / 2.06	5.34 / 3.95	**2.51** / 1.85	31.57	**1.88**

Table 2: Complete Chronological Log of All 13 Setbacks Encountered and Engineering Solutions Taken.

#	Stage	Setback Encountered	Root Cause	Solution Implemented
1	W1-07 Audit	Speaker `s21` missing video; 110 corrupt clips in `s8`, `s4`, `s28`	Raw GRID archive defects	Built `verify_dataset.py` to exclude `s21` and filter bad clips (32,890 valid kept)
2	W1-07 Sync	Audio lasted 1.1–2.9 s vs. 3.0 s video; voice ~ 0.47 s early	GRID `.wav` files were end-pointed (silence trimmed)	Shifted audio placement in `GridDataset` using `.align` word-onset timestamps
3	W2-01 Crop	MediaPipe missed faces on 1–2 frames in some clips	Brief motion blur or extreme lip posture	Linearly interpolated boxes from neighbour frames and marked `valid[t]=False` (kept $T = 75$)
4	W2-09 Align	Early transcripts decoded as numbers instead of words	`.align` parser read column `0` (time) instead of `2` (word)	Fixed `parse_align()` in `tokenizer.py`; verified on all 33,000 files
5	W2-09 RNG	DataLoader workers drew identical SNR sequences	Forked workers inherited identical RNG state	Reseeded RNG deterministically per `(seed, epoch, worker_id)`
6	W2-07 RMS	-5 dB mixtures were much louder than clean clips	-25 dBFS levelling ran *before* noise mixing	Moved RMS normalisation to run *after* noise mixing (gain $\leq +20$ dB)
7	W2-11 Git	`lipscribe/models/` files hidden from `git status`	Unanchored `models/` rule in `.gitignore`	Anchored rule to `/models/` in `.gitignore`
8	W3-05 Overfit	B2 overfit froze at CER = 0.2804, predicting `'bin blue at f sive snon'`	First 8 alphabetical clips shared `'bin blue at f '`; BiLSTM predicted suffix centroid	Sampled unigram-diverse verbs/colours (`load_overfit_batch`), hitting Loss = 0.0497, CER = 0.0
9	W3-05 B2 v1	2.5× WER gap across unseen speakers (13.3 % vs. 33.7 %)	Shallow 3-block `Conv3d` + `Linear(3456,512)` overfit to speaker geometry	Built `ResNetVisualEncoder` (`resnet18_bilstm`) with `input_norm="clip"` and `AdaptiveAvgPool2d(1)`
10	W3-06 B3 v1	B3 v1 beat B1 at -5 dB (7.35 %) but hurt clean speech (2.41 % vs. 1.96 %)	Fast-converging audio starved slow random-init visual stream	Added pre-trained checkpoint loading (`init_from`) + modality dropout $\rightarrow$ 1.16 % clean in B3 v2
11	W4-pre Ablat.	Dropping video or audio collapsed B3 v2 (25.18 % / 75.31 %)	Fixed linear concatenation lacks dynamic gating or fallback token	Built **Model P** with learned $\mathbf{e}_{\text{miss}}$, cross-attention, and gate $\mathbf{g}_t$ (1.88 % no-video, 23.78 % no-audio)
12	W4-02 Mask	Masking all keys in `MultiheadAttention` gave `NaN`	$\text{softmax}(-\infty, \ldots, -\infty) =$ NaN	Substituted learned $\mathbf{e}_{\text{miss}} \in \mathbb{R}^{256}$ *before* attention instead of masking all keys
13	W4-04 Eval	`evaluate.py` crashed with `ValueError: 'gate_mean'` on Model P	`save_metrics()` had fixed 5-column header without `gate_mean`	Updated `save_metrics()` in `scripts/evaluate.py` to include `gate_mean` dynamically

8 Chapter 8 — Team Command Cheat-Sheet: Run Everything Yourself

Listing 2: Complete Command Reference (Run from Repo Root on the Alienware Workstation)

```
# 1. Run the complete unit & integration test suite (pytest):
.venv/bin/python -m pytest -q

# 2. Run 8-utterance overfit sanity checks (< 2 minutes each):
.venv/bin/python -m scripts.check_b2_overfit
.venv/bin/python -m scripts.check_b3_overfit
.venv/bin/python -m scripts.check_p_overfit

# 3. Train any Baseline or Proposed Model P variant (inside tmux):
tmux new -s lipscribe
.venv/bin/python -m scripts.train --config configs/b1_audio_only.yaml
.venv/bin/python -m scripts.train --config configs/b2_visual_only.yaml
.venv/bin/python -m scripts.train --config configs/b3_concat_v2.yaml --time-budget-min 30
.venv/bin/python -m scripts.train --config configs/p_init_b1_b2_v2.yaml --time-budget-min 30

# 4. Evaluate our best Model P checkpoint (Greedy, Grammar, Beam Search, & Modality Ablations):
.venv/bin/python -m scripts.evaluate --checkpoint checkpoints/p_proposed_best.pt
```

```
.venv/bin/python -m scripts.evaluate --checkpoint checkpoints/p_proposed_best.pt --decode
    grammar
.venv/bin/python -m scripts.evaluate --checkpoint checkpoints/p_proposed_best.pt --decode beam
.venv/bin/python -m scripts.evaluate --checkpoint checkpoints/p_proposed_best.pt
    --drop-modality audio
.venv/bin/python -m scripts.evaluate --checkpoint checkpoints/p_proposed_best.pt
    --drop-modality visual

# 5. Regenerate all comparison plots and report figures:
.venv/bin/python -m scripts.plot_all_model_comparisons
```